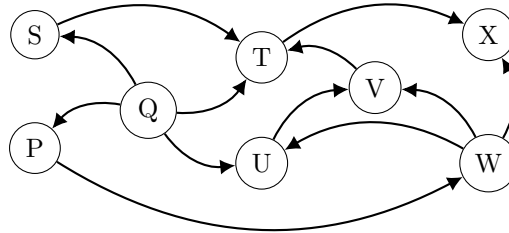


Week 10. Problem set (solutions by Karim Khabibrakhmanov)

1. Consider the following directed graph $\mathcal{G}_1$ and answer questions:



- (a) Write down **all** possible topological sortings for the vertices of $\mathcal{G}_1$.

Answer:

1.

Q	S	P	W	U	V	T	X
---	---	---	---	---	---	---	---
2.

Q	P	S	W	U	V	T	X
---	---	---	---	---	---	---	---
3.

Q	P	W	S	U	V	T	X
---	---	---	---	---	---	---	---
4.

Q	P	W	U	S	V	T	X
---	---	---	---	---	---	---	---
5.

Q	P	W	U	V	S	T	X
---	---	---	---	---	---	---	---

- (b) Write down **all** edges that can be safely removed from $\mathcal{G}_1$ without affecting the set of possible topological sortings.

Answer:

1.

Q	$\Rightarrow$	T
---	---------------	---
2.

Q	$\Rightarrow$	U
---	---------------	---
3.

W	$\Rightarrow$	V
---	---------------	---
4.

W	$\Rightarrow$	X
---	---------------	---

- (c) What is the maximum possible number of distinct topological sortings for the vertices of a graph with 8 vertices? Briefly justify your answer (1–2 sentences).

Answer: 40320

Let's imagine a directed Graph that has no connections at all. This satisfies topological sorting conditions, since Graph does not contain cycles. In fact, you can start from any vertex. The number of maximum different topological sorting with 8 vertices is equal to the number of permutations $P = 8! = 40320$.

- (d) What is the maximum possible number of distinct topological sortings for the vertices of a graph with 8 vertices and 12 edges? Briefly justify your answer (1–2 sentences).

Answer: 1440

In order to get the maximum number of topological sorting options, we need as many vertices in the graph as possible from which we can start (hereinafter referred to as independent vertices):

- 1) if we take 8 independent vertices, then we will not be able to apply 12 sides.
- 2) if we take 7 independent vertices and one in which everything converges, then we will have 7 sides. If you start adding sides so that there are 12, then the number of independent vertices will already decrease.
- 3) if we take 6 independent vertices, then we will have exactly 6 converging on each side in the remaining 2. It will be $2! \cdot 6! = 1440$. Into 6 different permutations with two permutations of dependent vertices.

There is no point in looking further since the number of independent vertices is only decreasing

2. Consider a large undirected graph $\mathcal{G}_2 = (V, E)$ where vertex labels are integers. We represent this graph with a variation of the adjacency-list graph representation [GTG, §14.2.2], where instead of lists we use maps, represented by B-trees [CLRS, §18] with minimum degree t and vertex labels

serving as keys in the search tree. What are the worst-case time complexities of the following Graph ADT operations over this modified representation, in terms of $|V|$, $|E|$, and t ?

- (a) `areAdjacent(v, u)`
Answer: $\Theta(\log_t(|V|))$
- (b) `insertVertex(v)`
Answer: $\Theta(1)$
- (c) `insertEdge(from, to, e)`
Answer: $\Theta(\log_t(|V|))$
- (d) `removeEdge(from, to)`
Answer: $\Theta(\log_t(|V|))$
- (e) `removeVertex(v)`
Answer: $\Theta(|V| \cdot \log_t(|V|))$

Assume that `v`, `from`, and `to` arguments above are *references* to vertex objects, not just labels. Briefly justify your answer (1–2 sentences).

Justification:

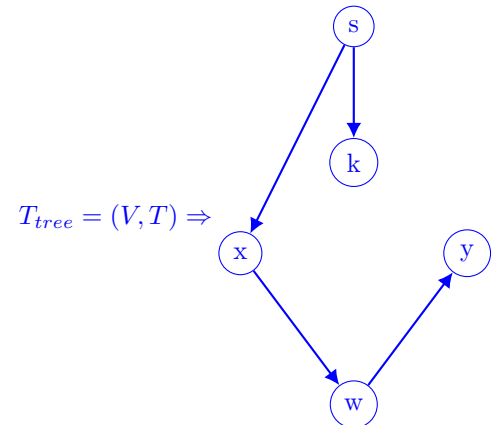
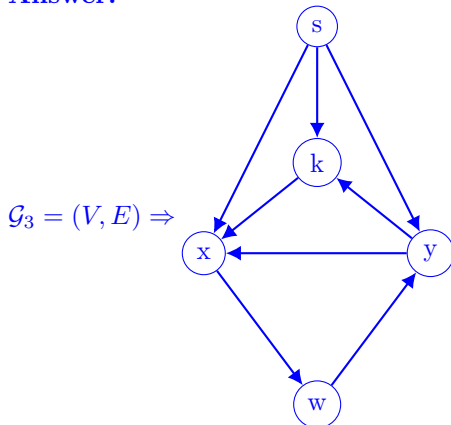
1. Since the links to the vertices are initially given, you can apply a B-tree search for the vertex v . In the worst case, it will take $\Theta(\log_t(|V|))$.
2. Adding a new vertex to the list is constant $\Theta(1)$.
3. Adding a side takes adding a new vertex to the B-tree for both vertices. In the worst case, it will take $\Theta(\log_t(|V|))$.
4. In fact, it is the same as adding, but only removing edges for each vertex $\Theta(\log_t(|V|))$.
5. In the worst case, we look through the list of vertices and for each vertex ($|V|$) we delete the edge associated with v in its B-tree for $\Theta(\log_t(|V|))$. Deleting a vertex will take $\Theta(|E| \cdot \log_t(|V|))$.

3. Provide a directed graph $\mathcal{G}_3 = (V, E)$, vertex $s \in V$, and a subset of edges $T \subseteq E$ such that

- both (V, E) and (V, T) are connected graphs,
- $|V| \leq 5$,
- T forms a tree with root s ,
- the set of edges T cannot be produced by running BFS [CLRS, §20.2] on $\mathcal{G}_3$, no matter how the vertices are ordered in the adjacency lists,
- the set of edges T cannot be produced by running DFS [CLRS, §20.3] on $\mathcal{G}_3$, no matter how the vertices are ordered in the adjacency lists.

Briefly justify your answer (1–2 sentences).

Answer:



Justification:

- (a) Both (V, E) and (V, T) are connected graphs.
- (b) $|V| = 5 \leq 5$.
- (c) T_{tree} forms a tree with root s .
- (d) It is impossible to get T_{tree} from graph $\mathcal{G}_3$ using **BFS** because at least there is no connection $\mathbf{s} \Rightarrow \mathbf{y}$. Since we start with $\mathbf{s}$, each adjacency vertex must be visited.
- (e) DFS cannot be applied. Let's try to represent T.
 - 1) If we start with $s \Rightarrow x \Rightarrow w \Rightarrow y \Rightarrow k$, our algorithm will end with $\mathbf{k}$, creating just a chain. We couldn't stop at $\mathbf{y}$ because we could go even deeper.
 - 2) If we start with $s \Rightarrow k \Rightarrow x \Rightarrow w \Rightarrow y$, our algorithm will end with $\mathbf{y}$, creating just a chain. We couldn't stop at $\mathbf{k}$ because we could go even deeper.

References

- [CLRS] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. *Introduction to algorithms, Fourth Edition*. MIT press.
- [GTG] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser. *Data Structures and Algorithms in Java*. WILEY 2014.